EEP Auto-Grading Platform

Setup & Deployment Guide

Complete step-by-step setup with troubleshooting

E-Yantra Summer Internship Program (EYSIP)

Covers **local development**, **production deployment**, **environment configuration**, and **troubleshooting** for the full EEP Auto-Grading stack.

```
Stack:  FastAPI + PostgreSQL + Redis + Celery + MinIO + Docker
Frontend:  React 18 + Vite + Tailwind CSS
Infra:  Railway (API) + AWS EC2 (Worker) + Vercel (Frontend)
```


Document Version: 1.0
Platform Version: v2.0
Audience: DevOps / Platform Administrator
Organisation: E-Yantra, IIT Bombay

Last updated: July 1, 2026

Contents

1 Architecture Overview

1.1 Hosting Split

Component	Host	Reason
PostgreSQL + Redis	Railway	Managed, low-latency to API
FastAPI API	Railway	Auto-scaling, zero-ops
React Frontend	Vercel	Global CDN, instant Git deploys
Celery Worker + MinIO	AWS EC2	Requires host Docker socket

Why EC2 for the Worker?

The Celery grading worker must **mount `/var/run/docker.sock`** to spawn isolated sibling containers per submission. PaaS platforms (Railway, Heroku) use containerized environments that do not expose the host Docker daemon. A plain EC2 VM provides direct host Docker socket access.

2 Prerequisites

2.1 Local Development Machine

Requirement	Notes
Docker Desktop 4.x+	Includes Docker Compose v2. https://www.docker.com/products/docker-desktop/
Node.js 18+	LTS recommended. https://nodejs.org
Python 3.10+	For local venv and seeding scripts
Git	For cloning and version control

2.2 Production / Cloud

Service	Purpose
Railway account	Hosts API + PostgreSQL + Redis
AWS account	EC2 instance for worker + MinIO
Vercel account	Hosts the React frontend
GitHub account	Source repository

3 Local Development Setup

3.1 Clone the Repository

1 Clone from GitHub

```
git clone https://github.com/RohitWakade07/v1.git eysip-
   platform
cd eysip-platform
```

3.2 Backend Setup

2 **Copy and configure the environment file**

```
cd backend
cp .env.example .env
# Open .env in your editor -- defaults work for local dev
# No changes required for local development
```

3 **Start all backend services with Docker Compose**

This spins up PostgreSQL, Redis, MinIO, the FastAPI API, and the Celery worker:

```
docker compose up -d --build
```

Expected output:

```
[+] Running 5/5
 [OK] Container grading_postgres  Started
 [OK] Container grading_redis     Started
 [OK] Container grading_minio     Started
 [OK] Container grading_backend   Started
 [OK] Container grading_worker    Started
```

4 **Verify all containers are healthy**

```
docker compose ps
```

All containers should show `running` or `healthy`.

5 **Run database migrations**

```
# Windows PowerShell:
$env:PYTHONPATH = "$(pwd)"
venv\Scripts\alembic upgrade head

# Linux / macOS:
PYTHONPATH=. alembic upgrade head
```

If you don't have alembic in a venv, run it inside the backend container:

```
docker compose exec backend alembic upgrade head
```

6 **Seed the initial admin account**

```
# Windows PowerShell:
$env:PYTHONPATH = "$(pwd)"; venv\Scripts\python seed_admin.py

# Linux / macOS:
PYTHONPATH=. python seed_admin.py
```

Expected output:

```
[seed_admin] Admin account created successfully!
  Username : eyantrastaff@gmail.com
  Password : eyantrastaff@gmail.com    <- CHANGE THIS
```

Change the Default Password

The default admin password is `eyantrastaff@gmail.com`. Log in and change it immediately.

3.3 Local Service Endpoints

Service	URL	Default Credentials
FastAPI Swagger Docs	http://localhost:8000/docs	– (DEBUG must be true)
MinIO Console	http://localhost:9001	minioadmin / minioadmin
PostgreSQL	`localhost:5433`	grading_user / changeme
Redis	`localhost:6379`	(no password)

3.4 Frontend Setup

Open a **new terminal** (keep backend running):

7 **Install dependencies**

```
cd frontend
npm install
```

8 **Configure environment**

```
# Create frontend .env.local
echo "VITE_API_BASE_URL=http://localhost:8000" > .env.local
```

9 **Start the development server**

```
npm run dev
```

Frontend runs at http://localhost:5173

4 Environment Variables Reference

4.1 Backend .env File

```
#  App
APP_NAME="E-Yantra EEP Platform"
DEBUG=false           # Set true only for local dev (enables /
   docs)
ENVIRONMENT=production

#  Database
POSTGRES_HOST=postgres
POSTGRES_PORT=5432
POSTGRES_USER=grading_user
POSTGRES_PASSWORD=changeme   # CHANGE IN PRODUCTION
POSTGRES_DB=grading_db

#  Redis
REDIS_HOST=localhost
REDIS_PORT=6379
REDIS_DB=0
REDIS_PASSWORD=              # Set if your Redis requires auth

#  JWT
# Generate: python -c "import secrets; print(secrets.token_hex
   (64))"
JWT_SECRET_KEY=CHANGE_THIS_BEFORE_DEPLOYING
JWT_ALGORITHM=HS256
JWT_ACCESS_TOKEN_EXPIRE_MINUTES=60

#  Proof Signing
PROOF_SIGNING_KEY=CHANGE_THIS_BEFORE_DEPLOYING

#  Evaluator
EVALUATOR_SHARED_KEY=        # Shared key for evaluator
   sessions

#  Rate Limiting
LOGIN_RATE_LIMIT_PER_MINUTE=10

#  Celery / Worker
CELERY_BROKER_URL=redis://localhost:6379/0
CELERY_RESULT_BACKEND=redis://localhost:6379/1

#  MinIO / Storage
MINIO_ENDPOINT=http://minio:9000
MINIO_ACCESS_KEY=minioadmin
MINIO_SECRET_KEY=minioadmin
MINIO_BUCKET_SUBMISSIONS=submissions
```

```
MINIO_BUCKET_ASSETS=grader-assets
MINIO_USE_SSL=false
MINIO_REGION=us-east-1

#  CORS
ALLOWED_ORIGINS=https://your-app.vercel.app

#  EEP Verifier
EEP_MAX_UPLOAD_BYTES=50000   # 50 KB max ZIP size

#  Kafka (optional, for future event streaming)
KAFKA_BOOTSTRAP_SERVERS=kafka:9092
```

4.2 Generating Secrets

```
python -c "import secrets; print(secrets.token_hex(64))"
```

Generate a unique value for **both** `JWT_SECRET_KEY` and `PROOF_SIGNING_KEY`.

5 Production Deployment

5.1 Phase 1: Database & API on Railway

1 Create a Railway project

Go to https://railway.app and create a new project.

2 Add PostgreSQL plugin

1. Click + **New** → **Database** → **PostgreSQL**.
2. After creation, copy the `DATABASE_URL` from the plugin's *Connect* tab.

3 Add Redis plugin

1. Click + **New** → **Database** → **Redis**.
2. Copy the Redis URL from the plugin's *Connect* tab.

4 Create the API service

1. Click + **New** → **GitHub Repo**.
2. Select your repository and set **Root Directory** to `backend/`.
3. Railway will auto-detect the Dockerfile and use `Dockerfile.api`.

5 Set environment variables in Railway

In the service settings → **Variables**, add:

```
# Railway PostgreSQL auto-provides this variable
DATABASE_URL=postgresql+asyncpg://...   # from Railway
   PostgreSQL plugin

# Redis
REDIS_HOST=<railway-redis-host>
REDIS_PORT=<railway-redis-port>
REDIS_PASSWORD=<railway-redis-password>
CELERY_BROKER_URL=redis://:<pass>@<host>:<port>/0
CELERY_RESULT_BACKEND=redis://:<pass>@<host>:<port>/1

# MinIO on EC2 (fill after Phase 2)
MINIO_ENDPOINT=http://<your-ec2-ip>:9000
MINIO_ACCESS_KEY=minioadmin
MINIO_SECRET_KEY=minioadmin
MINIO_BUCKET_SUBMISSIONS=submissions
MINIO_BUCKET_ASSETS=grader-assets
MINIO_USE_SSL=false

# Security (generate fresh values!)
JWT_SECRET_KEY=<generated-64-byte-hex>
PROOF_SIGNING_KEY=<generated-64-byte-hex>

# CORS
ALLOWED_ORIGINS=https://your-app.vercel.app

# App
DEBUG=false
ENVIRONMENT=production
```

6 Set the startup command

In Railway service settings → **Deploy** → **Start Command**:

```
alembic upgrade head && uvicorn app.main:app --host 0.0.0.0 --
   port 8000
```

7 Deploy

Railway will build and deploy automatically. Check the **Deployments** tab for logs.

API is Live

Visit `https://{your-railway-url}/health` – you should see `{"status":"ok"}`.

5.2 Phase 2: Celery Worker + MinIO on AWS EC2

5.2.1 Step 2a: Launch EC2 Instance

1 **Go to AWS Console → EC2 → Launch Instance**

Setting	Value
AMI	Ubuntu Server 24.04 LTS (64-bit, x86)
Instance Type	`m7i-flex.large` (8 GB RAM) — Minimum recommended *Avoid t3.small (2 GB is insufficient for Docker containers)*
Key Pair	Create or use existing `.pem` key
Security Group	Allow inbound: SSH (22), MinIO (9000), HTTP (80), HTTPS (443)
Storage	30 GB GP3 (minimum)

5.2.2 Step 2b: Install Docker on EC2

2 **SSH into your EC2 instance**

```
ssh -i "your-key.pem" ubuntu@<your-ec2-public-ip>
```

3 **Install Docker Engine**

```
# Update package index
sudo apt-get update

# Install dependencies
sudo apt-get install -y ca-certificates curl gnupg

# Add Docker GPG key
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
    sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg

# Add Docker repository
echo \
  "deb [arch=$(dpkg --print-architecture) \
  signed-by=/etc/apt/keyrings/docker.gpg] \
  https://download.docker.com/linux/ubuntu \
  $(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

# Install Docker CE + Compose plugin
sudo apt-get update
sudo apt-get install -y \
    docker-ce docker-ce-cli containerd.io \
    docker-buildx-plugin docker-compose-plugin git
```

4 **Add ubuntu user to Docker group**

```
sudo usermod -aG docker ubuntu
# IMPORTANT: Log out and log back in for this to take effect!
exit
ssh -i "your-key.pem" ubuntu@<ec2-ip>
```

5 **Verify Docker installation**

```
docker --version
# Docker version 26.x.x

docker compose version
# Docker Compose version v2.x.x
```

5.2.3 Step 2c: Clone and Configure

6 **Clone the repository**

```
git clone https://github.com/RohitWakade07/v1.git grading-
   platform
cd grading-platform/backend
```

7 **Create the worker `.env` file**

```
cp .env.example .env
nano .env
```

Configure the following (minimum required for EC2 worker):

```
SERVICE_TYPE=worker

#  From Railway PostgreSQL plugin
DATABASE_URL_OVERRIDE=postgresql+asyncpg://postgres:...@...
   railway.app:5432/railway

#  From Railway Redis plugin
REDIS_HOST=<railway-redis-host>
REDIS_PORT=<railway-redis-port>
REDIS_PASSWORD=<railway-redis-password>
CELERY_BROKER_URL=redis://:<pass>@<host>:<port>/0
CELERY_RESULT_BACKEND=redis://:<pass>@<host>:<port>/1

#  MinIO (running locally on this same EC2)
MINIO_ENDPOINT=http://localhost:9000
MINIO_ACCESS_KEY=minioadmin
MINIO_SECRET_KEY=minioadmin
MINIO_BUCKET_SUBMISSIONS=submissions
MINIO_BUCKET_ASSETS=grader-assets
MINIO_USE_SSL=false
```

```
#  SAME keys as Railway API
JWT_SECRET_KEY=<same-value-as-railway>
PROOF_SIGNING_KEY=<same-value-as-railway>
```

5.2.4 Step 2d: Start the Worker

8 **Build and start the worker**

```
cd grading-platform/backend
docker compose -f docker-compose.worker.yml up -d --build
```

9 **Verify the worker is running**

```
docker compose -f docker-compose.worker.yml ps
# grading_worker should show "running"

docker logs -f grading_worker
# Look for: celery@<id> ready.
```

10 **Update Railway with your EC2 IP for MinIO**

Go back to Railway and update:

```
MINIO_ENDPOINT=http://<your-ec2-public-ip>:9000
```

Redeploy the Railway service for this change to take effect.

5.3 Phase 3: Frontend on Vercel

1 **Import repository into Vercel**

1. Go to https://vercel.com/new and click **Import Git Repository**.
2. Select your GitHub repository.

2 **Configure Vercel project settings**

Setting	Value
Root Directory	`frontend`
Framework Preset	Vite
Build Command	`npm run build`
Output Directory	`dist`

3 **Add environment variable**

Under **Environment Variables**:

```
VITE_API_BASE_URL=https://<your-railway-api-url>
```

4 Deploy

Click **Deploy**. Vercel will build and deploy. Future pushes to your branch auto-deploy.

Deployment Complete!

Visit your Vercel URL. The full platform is now live.

6 Post-Deployment Steps

After a successful deployment, always perform these steps:

1 Seed the admin account (production)

```
# Via Railway Shell (in Railway dashboard -> Service -> Shell):
python seed_admin.py

# Or via railway CLI:
railway run python seed_admin.py
```

2 Seed weekly assignments

```
# Populate all weekly assignment definitions
railway run python seed_assignments.py
```

3 Verify the health endpoint

```
curl https://<your-railway-url>/health
# Expected: {"status":"ok","database":"ok","redis":"ok","
   version":"..."}
```

4 Test end-to-end flow

1. Login as admin, create a mentor account.
2. Login as mentor, create a classroom, import a student CSV.
3. Create and publish an assignment.
4. Login as student, make a test submission.
5. Verify the submission reaches COMPLETED state.

5 **Change the default admin password**

Log in as admin on the platform and update the password via the Profile page.

7 Updating the Platform

7.1 Updating the API (Railway)

Push changes to your linked branch. Railway auto-deploys on push.

If you added new database tables (new Alembic migrations), the startup command runs `alembic upgrade head` automatically before starting uvicorn.

7.2 Updating the Worker (EC2)

```
# SSH into EC2
ssh -i "your-key.pem" ubuntu@<ec2-ip>

cd grading-platform

# Pull latest code
git pull origin main

cd backend

# Rebuild and restart worker (zero-downtime not guaranteed)
docker compose -f docker-compose.worker.yml up -d --build

# Or just restart without rebuilding (for code-only changes):
docker compose -f docker-compose.worker.yml restart worker
```

7.3 Updating the Frontend (Vercel)

Push to your linked branch. Vercel auto-deploys.

8 Troubleshooting Guide

8.1 API Boot Loop / 405 Method Not Allowed

Symptoms

- FastAPI backend constantly restarts (Railway shows repeated deploys)
- Frontend API calls return `405 Method Not Allowed` or `502 Bad Gateway`
- Railway logs show `SyntaxError` or `IndentationError`

Root Cause: A Python syntax error in backend code causes Gunicorn/uvicorn to crash on startup.

Resolution:

1 **Identify the exact error in Railway logs**

```
# In Railway dashboard: Deployments -> View Logs
# Look for lines like:
IndentationError: unexpected indent in app/api/v1/routes/
    assignments.py, line 142
```

2 **Fix the syntax error locally and push**

```
# Validate before pushing
python -m py_compile backend/app/api/v1/routes/assignments.py

# Push the fix
git add .
git commit -m "fix: syntax error in assignments.py"
git push
```

3 **Emergency hotfix on EC2 (if worker is also affected)**

```
# Copy fix script into running worker container
docker cp hotfix.py $(docker compose ps -q worker):/app/hotfix.
    py
docker compose exec worker python /app/hotfix.py

# Restart the worker
docker compose restart worker
```

4 **Prevention**

```
# Add this to your pre-commit or CI pipeline:
python -m py_compile backend/app/api/v1/routes/*.py
```

8.2 Frontend TypeScript Build Failure

Symptoms

- Vercel deployment fails during build
- `npm run build` fails with `TS1002: Unterminated string literal`
- Error points to a component handling multi-line text

Root Cause: A multi-line string or template literal contains a raw newline instead of `\n`.

Resolution:

```
# Wrong:
assignment.expected_structure.split('
').map(...)

# Correct:
assignment.expected_structure.split('\n').map(...)
```

Test locally before pushing:

```
cd frontend
npm run build
```

8.3 Worker Not Picking Up Jobs

Symptoms

- Submissions stay stuck in QUEUED or PENDING state forever
- Student dashboard shows no status update after several minutes

Diagnosis:

```
# Check worker logs
docker logs -f grading_worker

# Check if Celery is connected to Redis
docker compose exec worker celery -A workers.tasks inspect ping

# Check Redis connectivity
docker compose exec worker redis-cli -h $REDIS_HOST -p
   $REDIS_PORT -a $REDIS_PASSWORD ping
# Expected: PONG
```

Common Fixes:

```
# 1. Restart the worker
docker compose -f docker-compose.worker.yml restart worker

# 2. If Redis URL changed, update .env and restart
docker compose -f docker-compose.worker.yml up -d
```

```
# 3. Check that CELERY_BROKER_URL in EC2 .env matches Railway
   Redis URL
grep CELERY_BROKER_URL .env
```

8.4 Database Migration Failure

Symptoms

- API crashes on startup with `UndefinedTable` or `OperationalError`
- `alembic upgrade head` fails with `Connection refused`
- Railway logs show `could not connect to server`

Diagnosis and Resolution:

```
# 1. Verify DATABASE_URL is set correctly in Railway
railway variables --service <your-api-service>

# 2. Run migrations manually via Railway shell
railway run alembic upgrade head

# 3. Check alembic revision history
railway run alembic history

# 4. If schema is out of sync, show current revision
railway run alembic current

# 5. For local development, check postgres is running
docker compose ps | grep postgres
# Should show: healthy

# 6. Connect directly to verify
docker compose exec postgres psql -U grading_user -d grading_db
    -c "\dt"
```

8.5 MinIO Connection / Storage Errors

Symptoms

- Submissions fail with `S3Error` or `Connection refused`
- Worker logs show `EndpointConnectionError`
- ZIP files are not stored, submissions get stuck

Diagnosis:

```
# Check MinIO is running on EC2
docker ps | grep minio
# Should show: grading_minio (Up ...)
```

```
# Test MinIO health from the worker container
docker compose exec worker curl http://localhost:9000/minio/
   health/live
# Expected: HTTP 200

# Verify EC2 Security Group allows port 9000 inbound
# AWS Console -> EC2 -> Security Groups -> Inbound Rules
# Must have: Custom TCP, Port 9000, Source 0.0.0.0/0
```

Fixes:

```
# 1. Restart MinIO if it crashed
docker restart grading_minio

# 2. Check buckets exist (create if missing)
docker run --rm -it --network=host minio/mc:latest \
  mc config host add local http://localhost:9000 minioadmin
     minioadmin

docker run --rm -it --network=host minio/mc:latest \
  mc mb local/submissions
docker run --rm -it --network=host minio/mc:latest \
  mc mb local/grader-assets

# 3. If MINIO_ENDPOINT in Railway .env points to wrong IP:
#    Update Railway env variable with correct EC2 public IP and
    redeploy
```

8.6 Docker Sandbox Fails to Start

Symptoms

- Worker logs show `docker: permission denied while trying to connect to the Docker daemon`
- Grading jobs fail immediately with a Docker error

Root Cause: The worker container cannot access the host Docker socket.

Resolution:

```
# Verify the socket is mounted in docker-compose.worker.yml:
# volumes:
#   - /var/run/docker.sock:/var/run/docker.sock

# Check socket permissions on the host
ls -la /var/run/docker.sock
# Expected: srw-rw---- ... root docker

# Add ubuntu user to docker group (if not already done)
sudo usermod -aG docker ubuntu
# Log out and log back in, then restart the worker
```

```
docker compose -f docker-compose.worker.yml down
docker compose -f docker-compose.worker.yml up -d
```

8.7 CORS Errors (Frontend Cannot Reach API)

Symptoms

- Browser console shows `CORS policy: No 'Access-Control-Allow-Origin' header`
- Frontend shows `Network Error` for all API calls

Resolution:

```
# In Railway API environment variables:
ALLOWED_ORIGINS=https://your-app.vercel.app

# For multiple origins, comma-separate:
ALLOWED_ORIGINS=https://your-app.vercel.app,https://custom-
   domain.com
```

Redeploy the API after updating this variable.

8.8 Login Fails / JWT Errors

Symptoms

- `401 Unauthorized` on all API calls after login
- `Invalid or expired token` error

Causes and Fixes:

```
# 1. JWT_SECRET_KEY mismatch between Railway and EC2
#    Verify both use IDENTICAL keys:
grep JWT_SECRET_KEY backend/.env          # EC2
# Compare with Railway environment variable

# 2. Token expired
#    Increase expiry in Railway:
JWT_ACCESS_TOKEN_EXPIRE_MINUTES=120

# 3. Clock skew between services (rare)
#    Sync NTP on EC2:
sudo timedatectl set-ntp true
```

8.9 Submission VALIDATION_ERROR

Symptoms

- Student submission immediately fails with VALIDATION_ERROR
- No grading is attempted

Common Causes:

Error Code	Cause
ZIP_REQUIRED	Source type is ZIP but no file was attached
INVALID_FILE_TYPE	Uploaded file is not a `.zip` archive
FILE_TOO_LARGE	ZIP exceeds 50,000 bytes
PATH_TRAVERSAL	ZIP contains paths with `../` (security block)
MISSING_FILES	Required files not found in ZIP structure

```
# Check the submission result logs for the exact error
GET /api/v1/submissions/{submission_id}/result
# stderr field will contain the rejection reason
```

8.10 Worker Code Changes Not Taking Effect

Symptoms

- You updated grading logic in `workers/tasks.py`
- The old grading behaviour persists

Root Cause: Celery workers do not hot-reload code changes.

Resolution:

```
# After pulling new code to EC2:
cd grading-platform/backend

git pull origin main

# Option A: Just restart (fast, uses current image)
docker compose -f docker-compose.worker.yml restart worker

# Option B: Rebuild image (for dependency or Dockerfile changes
   )
docker compose -f docker-compose.worker.yml up -d --build
```

9 Maintenance Procedures

9.1 Backing Up the Database

```
# From local (via port-forward or direct connection):
pg_dump -h localhost -p 5433 -U grading_user -d grading_db \
    -F c -f backup_$(date +%Y%m%d).dump

# Restore:
pg_restore -h localhost -p 5433 -U grading_user -d grading_db \
    backup_20250101.dump
```

9.2 Viewing Live Logs

```
# API logs (Railway dashboard or railway CLI):
railway logs --service <api-service-name>

# Worker logs (EC2):
docker logs -f grading_worker

# Celery task logs with timestamps:
docker logs grading_worker --timestamps --since 1h
```

9.3 Clearing the Submission Queue (Emergency)

```
# Connect to Redis and flush the Celery queue:
docker compose exec redis redis-cli FLUSHDB

# WARNING: This drops all pending jobs. Only use in emergencies
   .
```

9.4 Running Tests

```
cd backend

# Create and activate virtual environment
python -m venv venv

# Windows:
venv\Scripts\activate

# Linux/macOS:
source venv/bin/activate

pip install -r requirements.txt

# Run all backend tests (ensure services are running)
PYTHONPATH=. pytest tests/ -v

# Run a specific test file
PYTHONPATH=. pytest tests/test_submissions.py -v
```

10 Quick Reference Card

```
Start all local services:   docker compose up -d -build
Stop all local services:    docker compose down
Run migrations:             alembic upgrade head
Seed admin:                 python seed_admin.py
Seed assignments:           python seed_assignments.py
View API logs:              docker logs -f grading_backend
View worker logs:           docker logs -f grading_worker
Restart worker (EC2):       docker compose restart worker
Rebuild worker (EC2):       docker compose up -d -build
Health check:               curl {api-url}/health
Run tests:                  PYTHONPATH=.  pytest tests/ -v
```

Key URLs & Endpoints

Health:	GET /health
API Docs:	GET /docs (DEBUG=true only)
All submissions:	GET /api/v1/admin/submissions
All students:	GET /api/v1/admin/students
All mentors:	GET /api/v1/admin/mentors
Create mentor:	POST /api/v1/admin/mentors
Import students CSV:	POST /api/v1/mentor/students/csv
Submit assignment:	POST /api/v1/submissions
Stream SSE status:	GET /api/v1/submissions/{id}/status
Local MinIO Console:	http://localhost:9001